

Homework 14: Log Analysis

Analyse a log with stream pipelines, reach for a gatherer where `map` cannot go, and reproduce the parallel data race so you have watched it happen.

The Data

```
List.of(
    "INFO  auth    120",
    "ERROR auth    450",
    "INFO  billing 80",
    "WARN  billing 900",
    "ERROR auth    380",
    "INFO  search  60",
    "ERROR billing 1200",
    "INFO  auth    95",
    "WARN  search  700",
    "INFO  billing 110");
```

Parse each line into `record Entry(String level, String service, int millis)`.

Part One: Grouping

Create `LogAnalysis.java` in the classic form. Produce all of these, each as a **single** stream pipeline:

1. Entry count per service
2. Average response time per service
3. The distinct levels seen per service, sorted
4. A two-way split of entries into slow (over 500ms) and fast
5. The slowest time per service

Requirements:

- Every grouped result must print in a **predictable** key order. Say in a comment which map type you used and why the default was not acceptable.
- Use a downstream collector for at least three of them, not a second pass.
- For the two-way split, use `partitioningBy` and write a comment on what it guarantees that `groupingBy` on a boolean does not.
- For the slowest time, first try a plain `toMap` and record what happens, then fix it.

Part Two: Gatherers

Extract just the timings, then produce:

1. Batches of three
2. The difference between each consecutive pair of readings
3. A running total

The Question

Write a comment explaining, in your own words, why **none** of these three can be written with `map`. Your answer needs to address all three, and they fail the same test for different reasons. Name the rule `map` obeys and say which part each one breaks.

Part Three: Race It

Add 50,000 integers to a plain `ArrayList` from a parallel stream. Do it **three times** and print the resulting size each run.

Guard it with a try/catch, because losing elements is not the only failure mode.

Then produce the correct count two ways: with `.toList()` and with `.count()`.

Explain It

Write a comment covering:

1. **The mechanism.** `ArrayList.add` is more than one step. Name the steps and say what two threads interleaving can do to each other.
2. **Why `collect` is immune.** What does it do differently with the containers?
3. **Why adding `synchronized` would be the wrong fix**, even though it would be correct.

Part Four: Laziness and Single Use

Build a pipeline with a `peek` in it but no terminal operation, and show that nothing runs.

Add `findFirst`, and show from the peek output that it **stopped early** rather than examining all ten entries.

Then call a second terminal operation on the same stream variable, catch what happens, and print the message. Finish by getting the same answer correctly from the source.

Acceptance Criteria

- ☐ Runs with `java LogAnalysis.java`
- ☐ All five part-one results come from single pipelines
- ☐ Key order is predictable, with a comment justifying the map type

- ☐ At least three downstream collectors are used
- ☐ A comment states what `partitioningBy` guarantees
- ☐ The `toMap` duplicate-key failure is triggered and then fixed
- ☐ All three gatherer operations produce correct output
- ☐ The comment explains why `map` cannot do any of the three
- ☐ Three parallel runs print their sizes, guarded against throwing
- ☐ The correct count is shown via `.toList()` and `.count()`
- ☐ The race explanation covers mechanism, why `collect` is safe, and why a lock is the wrong fix
- ☐ The peek output proves both laziness and short-circuiting
- ☐ Stream reuse is caught and its message printed

Stretch

Replace the parallel stream in part three with `Gatherers.mapConcurrent(4, ...)`.

Then write a comment comparing the two. Which one bounds its concurrency? Which preserves order? Which one would you use for work that calls a network service, and what does Module 14 say about why the other is a bad idea there?

Hint, if your deltas come out one element too long

`windowSliding(2)` over ten elements yields nine windows, not ten. There are nine gaps between ten readings. If you got ten, check whether you are windowing or mapping.